

surgic: local AI document redaction

A pipeline that redacts restricted documents with a local AI model on one air-gapped Mac Studio. No cloud services are involved. Documents are read from a read-only SMB share, processed only in RAM, and written back to a second share with signed proof that nothing left the machine.

Built and tested: about 210 automated tests, CI on macOS and network-disabled Linux

Not yet run on the target Mac Studio

For InfoSec and Cloud Engineering review October 2026

Decisions requested from InfoSec

- Accept the software-protected signing key for the pilot, or require the Secure Enclave option.
- Approve the model, Qwen 3.6 27B (`qwen3.6:27b`), pinned to the exact weight digest recorded at provisioning (checked by `--expect-model`).
- Set the human-review sampling rate for released documents.

Requested from Cloud and Infrastructure

- A read-only input share and a writable output share on one SMB host, reachable by IP, with SMB 3 encryption required.
- A dedicated VLAN with a static IP for the Mac Studio, carrying only SMB.
- A service account with access to both shares and nothing else.
- Jamf: AirDrop, Handoff and AirPlay Receiver disabled; Wi-Fi and Bluetooth off; FileVault on.

How a document moves

IN	WORKSPACE	STEP 1	STEP 2	STEP 3	STEP 4	OUT
SMB share, read-only	RAM disk only	Sandboxed: rules (regex, Presidio)	Local LLM on masked text	Sandboxed: redact the file	Separate sandbox: re-scan	SMB share, after the whole run

A document that fails the re-scan is quarantined; if the run aborts, nothing is released.

Controls and evidence

REQUIREMENT	HOW IT'S ENFORCED	EVIDENCE ON THE OUTPUT SHARE
No network egress	The firewall (pf) blocks everything except encrypted SMB to the one share host, on the SMB VLAN interface. Preflight checks the loaded rules exactly and the route to the host. A test probe to an outside address must be blocked.	<code>egress_audit.pcap</code> (all interfaces) holds <code>0 packets</code> ; <code>pflog_blocked.pcap</code> shows the blocks; both headers only
No data persistence	All working files live on a RAM disk, which is zero-filled and detached at the end; until it is, the firewall stays up. Core dumps are off and swap is encrypted.	Signed <code>closure.json</code> listing each teardown step and its exit code
Parsers can't reach the key, network or root	Every file parser runs in a sandboxed worker: no network, no Keychain, no sudo, writes only to the RAM disk. A second worker that never sees the input re-scans the output.	<code>isolation: sandbox</code> in the manifest

REQUIREMENT	HOW IT'S ENFORCED	EVIDENCE ON THE OUTPUT SHARE
No cross-document leakage	Model context is purged for every document, and the model is unloaded after each batch. An unload that can't be verified aborts the run.	Unload and reset counts in the manifest, checked by the verifier
Structured data never reaches the LLM	Regex and Presidio redact first. The LLM sees placeholders such as [US_SSN_1] , never the values.	Per-document counts in the manifest; values stored only as one-time HMACs
Nothing hidden survives	PDF layers, cropped and off-page content are exposed and every page is OCR'd in full; hidden sheets and rows are unhidden. Text files carrying encoded blobs are quarantined.	Per-document scan results
Output is actually clean	Output files are re-extracted with full-page OCR and checked for leftover values, then re-run through the detectors, gitleaks and trufflehog.	Per-document scan results; SHA-256 of every input and output

```
pass out quick on <SMB_IF> inet proto tcp from any to <SMB_HOST_IP> port 445 flags S/SA keep state
```

Above: the firewall's only pass rule.

Manifests and closure records are signed (Ed25519); logs hold no document text or file names.

How to verify a run

On a separate workstation, using your own copy of the public key:

```
surgic verify manifests/<run>.manifest.json --pubkey pubkey.pem --outputs <share> \
--closure evidence/<ts>/closure.json --expect-model <sha256>
surgic verify evidence/<ts>/closure.json --pubkey pubkey.pem
```

VERIFIED confirms all of the following:

- ✓ Signatures are valid
- ✓ Every output hash matches; no unlisted files
- ✓ Nothing quarantined was released; run not aborted
- ✓ Preflight passed; no control weakened
- ✓ Manifest belongs to the airgap session
- ✓ Egress capture holds 0 packets
- ✓ Firewall logged blocks
- ✓ RAM disk was wiped

Fail-closed behavior

- Preflight checks firewall, SMB route and encryption, radios, listeners, RAM disk, sandbox and model hash.
- Any failed check stops the run before a document is read.
- Any error on a document quarantines it, and nothing from it is written.
- Test-only shortcuts need an explicit opt-in flag, and the verifier rejects runs that used one.

Residual risks

1 The signing key is software-protected

It's stored in the login Keychain, so someone with the user's login could export it. Sandboxed parsers can't reach it.

Mitigation available: a Secure Enclave key via Jamf (in progress).

2 LLM recall is imperfect

Unstructured secrets depend on model quality. In testing, Qwen 3.6 27B caught contextual secrets in prose but missed a client name in a spreadsheet cell.

Recommended: human spot-checks when a new document type is introduced.

3 Prompt injection

A document could try to steer the model. The worst case is under-redaction, never exfiltration.

5 Memory pressure could cause swapping

Swap is encrypted with an ephemeral key, and the model (about 17 GB) leaves ample headroom.

7 A sandbox escape chained with a second exploit

Could cause under-redaction; neither worker can reach the network or the key.

9 To confirm on the target Mac

LibreOffice in the sandbox, `smbutil` encryption fields, and the `pktap, all` capture format.

4 OCR limits

Text that OCR can't read, such as poor handwriting, can't be detected.

6 Layer-2 traffic isn't filtered by the firewall

This covers ARP and IPv6 neighbor discovery.

Requires: a static IP on a dedicated VLAN.

8 Input hashes are recorded

Anyone holding a candidate low-variety document could confirm it was processed.

Source: docs/INFOSEC.md and spec.md in the surgic repository.

Rebuild this brief and its PDF with `scripts/build_brief.zsh`.